

A Practical Guide to Bagging ensemble

TOPIC -- BAGGING ENSEMBLE

-- 01 --

Creating the out-of-bag dataset The out-of-bag dataset represents the remaining people who were not in the bootstrap dataset. It can be calculated by taking the difference between the original and the bootstrap datasets. In this case, the remaining samples who were not selected are Emily, Jessie, George, Rachel, and Jamal. Keep in mind that since both datasets are sets, when taking the difference the duplicate names are ignored in the bootstrap dataset. The illustration below shows how the math is done:

-- 02 --

For a weak learner with high bias, bagging will also carry high bias into its aggregate Loss of interpretability of a model. Can be computationally expensive depending on the dataset.

-- 03 --

For classification, use a training set D , Inducer I and the number of bootstrap samples m as input. Generate a classifier C^* as output

-- 04 --

Specify the maximum depth of trees: Instead of allowing the random forest to continue until all nodes are pure, it is better to cut it off at a certain point in order to further decrease chances of overfitting. Prune the dataset: Using an extremely large dataset may create results that are less indicative of the data provided than a smaller set that more accurately represents what is being focused on. Continue pruning the data at each node split rather than just in the original bagging process. Decide on accuracy or speed: Depending on the desired results, increasing or decreasing the number of trees within the forest can help. Increasing the number of trees generally provides more accurate results while decreasing the number of trees will provide quicker results.

-- 05 --

Suppose the original dataset is a group of 12 people. Their names are Emily, Jessie, George, Constantine, Lexi, Theodore, John, James, Rachel, Anthony,

Ellie, and Jamal. By randomly picking a group of names, let us say our bootstrap dataset had James, Ellie, Constantine, Lexi, John, Constantine, Theodore, Constantine, Anthony, Lexi, Constantine, and Theodore. In this case, the bootstrap sample contained four duplicates for Constantine, and two duplicates for Lexi, and Theodore.

-- 06 --

Many weak learners aggregated typically outperform a single learner over the entire set, and have less overfit Reduces variance in high-variance low-bias weak learner, which can improve efficiency (statistics) Can be performed in parallel, as each separate bootstrap can be processed on its own before aggregation. Disadvantages:

-- 07 --

Key Terms There are three types of datasets in bootstrap aggregating. These are the original, bootstrap, and out-of-bag datasets. Each section below will explain how each dataset is made except for the original dataset. The original dataset is whatever information is given.

-- 08 --

Further reading Breiman, Leo (1996). "Bagging predictors". Machine Learning. 24 (2): 123–140. CiteSeerX 10.1.1.32.9399. doi:10.1007/BF00058655. S2CID 47328136. Alfaro, E., Gámez, M. and García, N. (2012). "adabag: An R package for classification with AdaBoost.M1, AdaBoost-SAMME and Bagging". {{cite journal}}: Cite journal requires |journal= (help)CS1 maint: multiple names: authors list (link) Kotsiantis, Sotiris (2014). "Bagging and boosting variants for handling classifications problems: a survey". Knowledge Eng. Review. 29 (1): 78–100. doi:10.1017/S0269888913000313. S2CID 27301684. Boehmke, Bradley; Greenwell, Brandon (2019). "Bagging". Hands-On Machine Learning with R. Chapman & Hall. pp. 191–202. ISBN 978-1-138-49568-5.

-- 09 --

Improving Random Forests and Bagging While the techniques described above utilize random forests and bagging (otherwise known as bootstrapping), there are certain techniques that can be used in order to improve their execution and voting time, their prediction accuracy, and their overall performance. The following are key steps in creating an efficient random forest:

-- 10 --

Bagging leads to "improvements for unstable procedures", which include, for example, artificial neural networks, classification and regression trees, and subset selection in linear regression. Bagging was shown to improve preimage learning. On the other hand, it can mildly degrade the performance of stable methods such as k-nearest neighbors.

-- 11 --

Example: ozone data To illustrate the basic principles of bagging, below is an analysis on the relationship between ozone and temperature (data from Rousseeuw and Leroy (1986), analysis done in R). The relationship between temperature and ozone appears to be nonlinear in this dataset, based on the scatter plot. To mathematically describe this relationship, LOESS smoothers (with bandwidth 0.5) are used. Rather than building a single smoother for the complete dataset, 100 bootstrap samples were drawn. Each sample is composed of a random subset of the original data and maintains a semblance of the master set's distribution and variability. For each bootstrap sample, a LOESS smoother was fit. Predictions from these 100 smoothers were then made across the range of the data. The black lines represent these initial predictions. The lines lack agreement in their predictions and tend to overfit their data points: evident by the wobbly flow of the lines.

-- 12 --

Create m new training sets D_i , from D with replacement Classifier C_i is built from each set D_i using I to determine the classification of set D_i

-- 13 --

Random Forests The next part of the algorithm involves introducing yet another element of variability amongst the bootstrapped trees. In addition to each tree only examining a bootstrapped set of samples, only a small but consistent number of unique features are considered when ranking them as classifiers. This means that each tree only knows about the data pertaining to a small constant number of features, and a variable number of samples that is less than or equal to that of the original dataset. Consequently, the trees are more likely to return a wider array of answers, derived from more diverse knowledge. This results in a random forest, which possesses numerous benefits over a single decision tree generated without randomness. In a random forest, each tree "votes" on whether or not to classify a sample as positive based on its features. The sample is then classified based on majority vote. An example of this is given in the diagram

below, where the four trees in a random forest vote on whether or not a patient with mutations A, B, F, and G has cancer. Since three out of four trees vote yes, the patient is then classified as cancer positive.